

Objectifs :

- Encapsuler la création d'objets complexes
- Encapsuler les algorithmes
- Mettre à profit le polymorphisme
- Penser à l'évolutivité/modularité du code

1 TP2 — Librairie de gestion de graphes

Objectifs pédagogiques

Ce TP a pour objectif de vous familiariser avec :

- L'héritage et le polymorphisme en Java ;
- La conception orientée objet (classes abstraites, encapsulation, packages) ;
- L'utilisation de tableaux et collections ;
- L'implémentation d'algorithmes sur les graphes.

Contexte

Vous devez développer une **librairie Java modulaire** permettant de représenter et d'analyser différents types de graphes. L'objectif est de construire une architecture claire, extensible et réutilisable.

Cette librairie devra permettre :

- de modéliser des graphes orientés ou non orientés ;
- d'exécuter plusieurs algorithmes de base : parcours, degré, plus courts chemins, etc. ;
- de pouvoir ajouter facilement de nouveaux algorithmes grâce au polymorphisme.

Exigences fonctionnelles

1. Création et manipulation de graphes La librairie doit permettre :

- de créer un graphe composé de sommets (*nodes*) et d'arêtes (*edges*) ;
- de choisir entre un graphe orienté ou non orienté ;
- d'ajouter et supprimer des sommets ;
- d'ajouter et supprimer des arêtes.

2. Contraintes à respecter Pour garantir la cohérence des graphes, les opérations suivantes sont interdites :

- les boucles (une arête reliant un sommet à lui-même) ;
- les multi-arêtes (deux arêtes identiques entre les mêmes sommets).

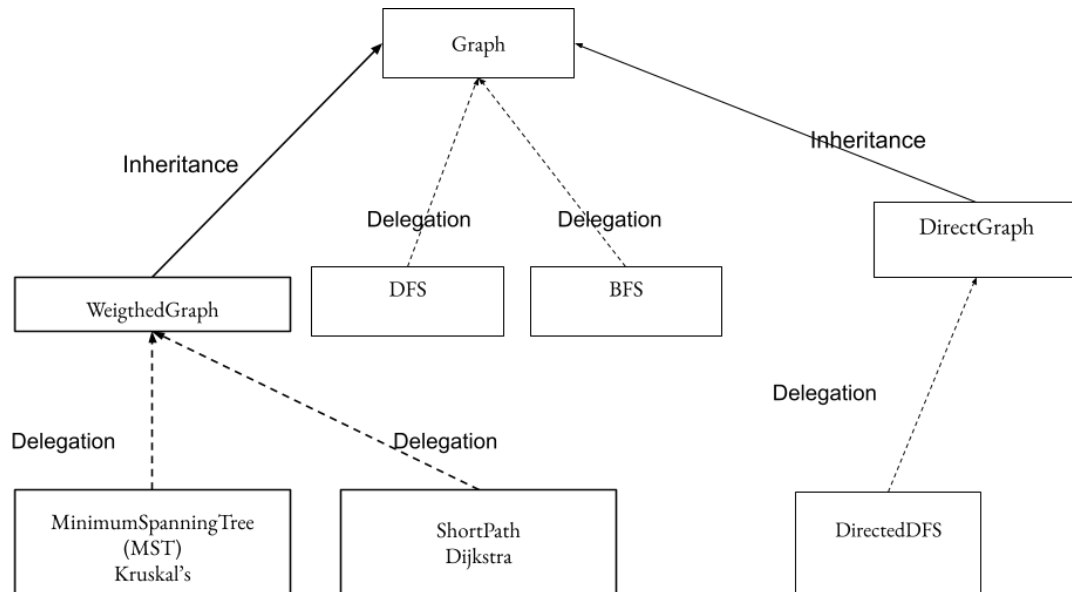
3. Algorithmes sur les graphes La librairie doit permettre d'exécuter différents algorithmes, notamment :

- parcours en profondeur (DFS) ;
- parcours en largeur (BFS) ;
- calcul du degré d'un sommet ;
- plus courts chemins (version simple) ;
- détection du voisinage (*neighbours*).

Les algorithmes doivent être conçus pour être **facilement extensibles** via l'héritage.

Diagramme UML de référence

Le diagramme suivant illustre une architecture possible pour structurer votre librairie.



Il constitue un **point de départ**, que vous pouvez adapter et enrichir si nécessaire.

4. Lecture et écriture de graphes

Votre librairie doit permettre d'importer et exporter des graphes depuis ou vers un fichier sous forme de :

- liste d'arêtes ;
- liste d'adjacence.

Travail demandé

1. Concevoir le **diagramme de classes** complet de votre librairie en détaillant :
 - les attributs ;
 - les méthodes ;
 - les relations (composition, héritage, dépendances).
2. Implémenter les classes correspondant au diagramme choisi, en respectant :
 - l'encapsulation ;
 - le découpage en packages ;
 - la documentation JavaDoc.
3. Implémenter les premiers algorithmes simples (au moins 2 parmi BFS, DFS, degré...).
4. Mettre en place un dépôt GitLab (université) contenant :
 - votre code source ;
 - un fichier README.md décrivant l'avancement du projet ;
 - le diagramme UML ;
 - un suivi des séances (document fourni).
5. Mettre à jour le dépôt à chaque fin de séance.